

Data Science

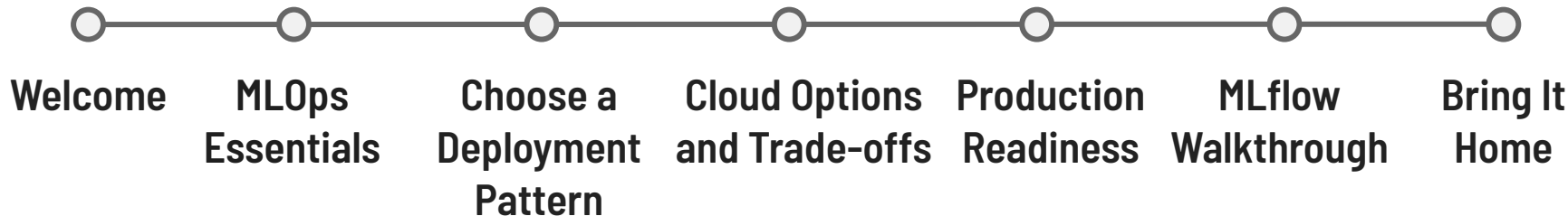
Deploying and Operating Machine Learning Models

Introduction to Data Engineering and MLOps



GENERAL ASSEMBLY

Lesson roadmap



Learning objectives



1

Explain what MLOps is and its role in the data science lifecycle.

2

Choose an appropriate deployment approach.

3

Apply core MLOps best practices.

4

Deploy and test a model locally with MLflow.

Your model works in Jupyter. What happens next?

You have trained a model, evaluated it on test data, and decided its performance is good enough to use.

- ☐ What else must be true before another application or business process can safely depend on it?



MLOps Essentials

A good model is not yet a production system

MODEL DEVELOPMENT

Build and evaluate a model in a development environment.

Focus on data, features, algorithms, and predictive performance.

Produce a fitted model or pipeline.

MODEL IN PRODUCTION

Make predictions available to another user, application, or process.

Preserve the same preprocessing used during training.

Handle valid and invalid requests reliably.

Record, monitor, maintain, and replace model versions.

Predictive performance is necessary but not sufficient to have a model ready to use in production.



What MLOps adds

MLOps is the practice of making machine learning workflows repeatable, controlled, deployable, and maintainable.

It helps teams:

- Reproduce how a model was created.
- Package the complete prediction workflow.
- Test and release versions consistently.
- Monitor models after release.
- Update or roll back models safely.

**Deployment makes a model available.
MLOps manages how it is operated over time.**





Choose a Deployment Pattern

There are two core deployment patterns

BATCH INFERENCE

Predict many records together

Usually scheduled or triggered

Higher latency is acceptable

Efficient for large, predictable workloads

No need for always-on capacity

ONLINE INFERENCE

Predict one request at a time

Available through a live endpoint

Low latency matters

Suited to live applications and decisions

Often requires ready and scalable capacity

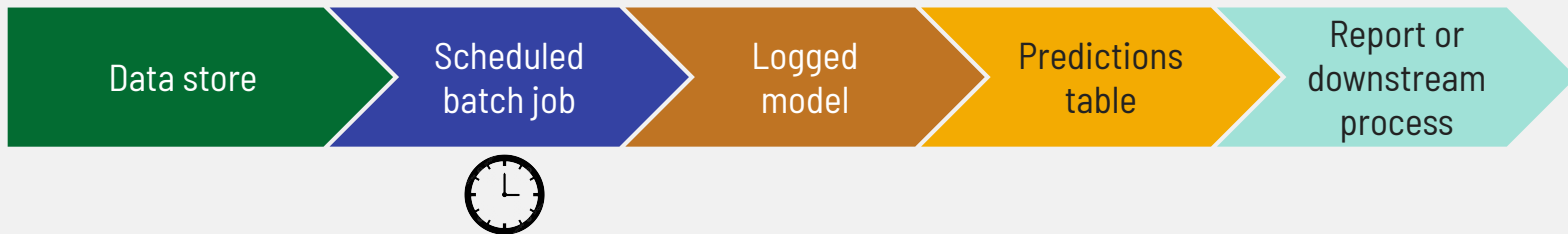
Batch inference

Use **batch inference** when:

- ❑ Predictions do not need to be immediate.
- ❑ Many records are processed together.
- ❑ Processing happens on a schedule.
- ❑ Workloads are predictable.
- ❑ Avoiding always-on compute is important.

Examples:

- Overnight delivery-risk scores
- Weekly customer segments
- Monthly demand forecasts



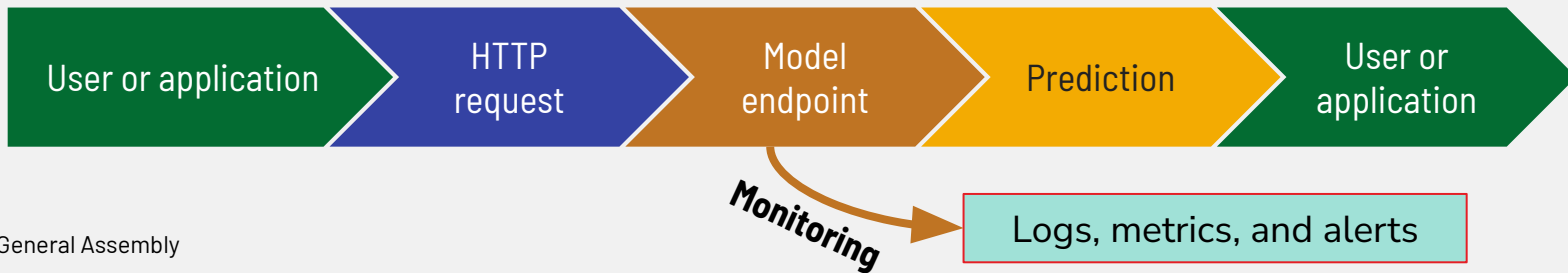
Online inference

Use **online inference** when:

- ❑ A prediction is needed during a live process.
- ❑ Requests arrive individually or continuously.
- ❑ Response time matters.
- ❑ Another application needs an API.
- ❑ The service must scale with traffic.

Examples:

- Fraud checks
- Routing decisions
- Product recommendations



Choose the operating pattern first

Ask these questions:

1. How quickly is the prediction needed?
2. How many predictions are required?
3. Is usage scheduled, steady, or unpredictable?
4. What happens if the model is unavailable?
5. What data-location or security constraints apply?
6. How much idle compute is acceptable?

**Start with the operating requirement rather than a
cloud-provider's product**



Choose the Pattern

For each scenario here, in pairs choose **Batch** or **Online** and identify the most important trade-off:

- A. Generate delivery-risk scores for two million packages every night.
- B. Flag a potentially incorrect routing decision as soon as a parcel is scanned.
- C. Run an analyst's forecasting model once each quarter for an internal planning meeting.

☐ **Discuss:** What would make you change your answer?

Choose the Pattern - Solution

SCENARIO	BATCH OR ONLINE...? AND THE MOST IMPORTANT TRADE-OFF...?
A. Overnight package-risk scoring	Batch: high volume, scheduled processing, and no requirement for individual low-latency responses.
B. Live routing check	Online: the prediction is needed inside a live operational decision and response time matters.
C. Quarterly forecast	Batch or local execution: the low frequency and small audience do not justify an always-on endpoint.



The same model could be deployed differently when the operating context changes.



Cloud Options and Trade-offs

Cloud deployment map

NEED	AWS	AZURE	GCP
Managed Online Inference	SageMaker AI Real-Time Inference	Azure ML Managed Online Endpoints	Vertex AI Endpoints
Managed Batch Inference	SageMaker AI Batch Transform	Azure ML Batch Endpoints	Vertex AI Batch Prediction
Model Management	SageMaker model capabilities	Azure ML registries	Vertex AI Model Registry
Production Monitoring	SageMaker monitoring capabilities	Azure ML monitoring / Azure Monitor	Vertex AI Model Monitoring



Cost and Performance Trade-offs

COST AND PERFORMANCE

- Ready capacity reduces latency but may increase idle cost.
- More replicas improve capacity and resilience but increase cost.
- Larger models or accelerators may improve capability but cost more to run.
- Batch processing can improve efficiency but delays individual responses.

The most accurate model is not automatically the best model to deploy.



A black and white photograph of a train station platform. Several people are walking away from the camera, their figures silhouetted against the bright light coming from the tracks. The floor is wet and reflective. A red vertical bar is on the left side of the image.

Production Readiness

Production-Readiness Check

PRODUCTION-READINESS CHECK

- **Reproducible:** Can we identify the pipeline, parameters, dependencies, and model version?
- **Valid:** Are expected inputs defined and tested?
- **Observable:** Can we detect errors, latency problems, data changes, and performance decline?
- **Recoverable:** Can we return to a previous working version?
- **Owned:** Is responsibility for monitoring and maintenance clear?

Note: When live input data starts to look different from the data a model was trained on, this is called **(model) drift**. It's one of the most common reasons an accurate model quietly gets worse over time, which is why **Observable** matters even for a model that passed all its tests.



Production-Readiness

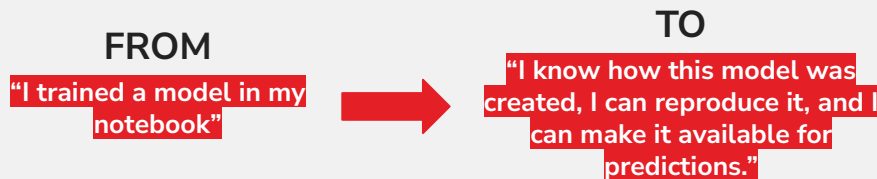
Return to one scenario from the **Choose the Pattern** exercise you just did with your partner. For that scenario, recommend:

- ☐ **Deployment:** Batch or online?
- ☐ **Trade-off:** What is the most important cost or reliability consideration?
- ☐ **Monitor:** What is one thing you would monitor?
- ☐ **Respond:** What would trigger rollback or retraining?

A high-contrast, black and white photograph of a train station platform. The silhouettes of several people are visible as they walk away from the camera towards a bright light source at the end of the platform. The floor is wet and reflective, mirroring the figures and the light. The overall mood is dramatic and cinematic.

MLflow Walkthrough

What is MLflow?



MLflow is a platform for managing machine learning models across their lifecycle.

In this lesson, we will use MLflow to:

- ❑ **TRACK:** Record model parameters, metrics, and other information about each modelling run.
- ❑ **PACKAGE:** Save the trained pipeline together with information about the inputs and outputs it expects.
- ❑ **SERVE:** Expose the saved model through an endpoint so another application can request predictions
- ❑ **REGISTER:** Promote a chosen model version into a central registry so the team always knows which version is live and can roll back to a previous one if needed.



Deploying the complete pipeline

Package preprocessing and prediction together so training and inference use the same logic.

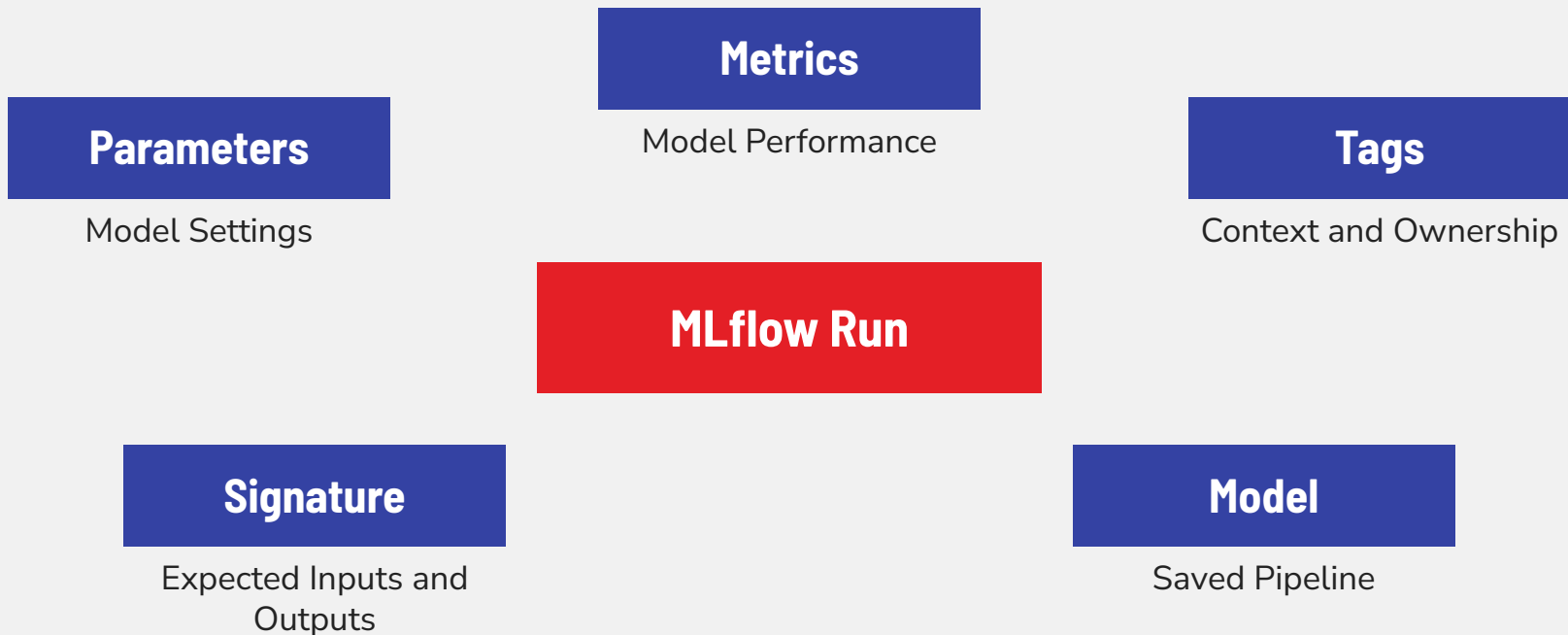
RISKY DEPLOYMENT



TRAINING WORKFLOW



What does an MLflow run record?



Recording this information makes models easier to compare, reproduce, and deploy.

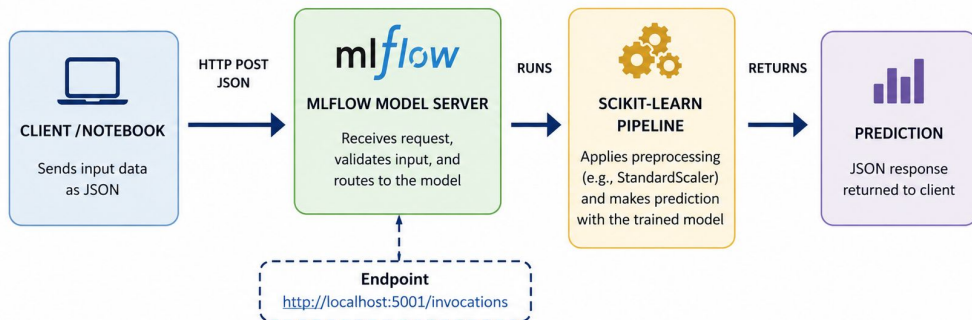
Serving and testing the model locally

mlflow models serve turns the logged model into a local prediction endpoint.

Architecture: Local Model Serving with MLflow

A client sends data to the MLflow model server.

The server runs the trained pipeline and returns a prediction.



Key Point:

Any application that can send an HTTP request can use this model endpoint to get predictions.

Test three things:

1. The health endpoint responds.
2. A valid request returns a prediction.
3. An invalid request returns a clear error.

Example request:

```
{"dataframe_split": {  
  
  "columns": ["sbp", "tobacco", "ldl",  
    "adiposity", "famhist", "typea",  
    "obesity", "alcohol", "age"],  
  
  "data": [[127, 0.0, 2.81, 15.7, 0, 42,  
    22.03, 1.03, 17]] }}
```

Local serving proves the deployment path.
Production serving requires additional security,
scaling, monitoring, and operational controls.

Track, Package, Serve, and Test with MLflow

Let's see it for ourselves!

- Open **01-mlflow-guided-walkthrough.ipynb**



By the end of this walkthrough, you will be able to:

1. Find the experiment and run in the MLflow UI.
2. Locate the parameters, accuracy, ROC AUC, and tags.
3. Open the logged model artifact.
4. Find the input example and model signature.
5. Reload the model using its MLflow model URI.
6. Verify that the reloaded model produces the same predictions.

As you work, consider: Which recorded item would be most useful during an incident six months after deployment?

Now You Try: log and compare a candidate

Open **02-mlflow-now-you-try.ipynb**

Your challenge:

1. Change one meaningful model setting.
2. Log a second candidate run in the same experiment.
3. Compare its parameters and metrics with the baseline.
4. Recommend which model should be deployed.
5. Identify one monitoring metric.
6. Identify one rollback or retraining trigger.

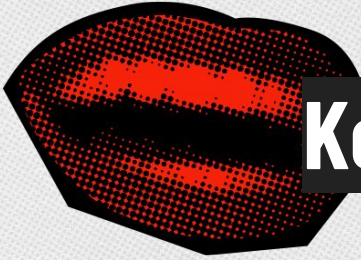


Optional stretch: Register the selected model or send an invalid request to the already running endpoint.

Success criteria:

- Two visible MLflow runs.
- Parameters, metrics, and tags for both runs.
- A justified deployment recommendation.
- One monitoring signal and one recovery trigger.

Bring It Home



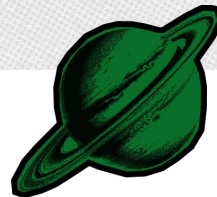
Key takeaways



**MLOps helps make
models reproducible,
deployable, and
maintainable.**



**Choose deployment
approaches based on
latency, scale, cost,
and reliability needs.**



**MLflow can track,
package, serve, and
test machine learning
models.**

